SECTION 04 · LOCAL RUNTIME · OPEN DECISION MODELS

Laya on Your Machine with Ollaya

Run an open decision model on your own computer: the same questions and the same SDK code as hosted Jev, with no text leaving the machine.

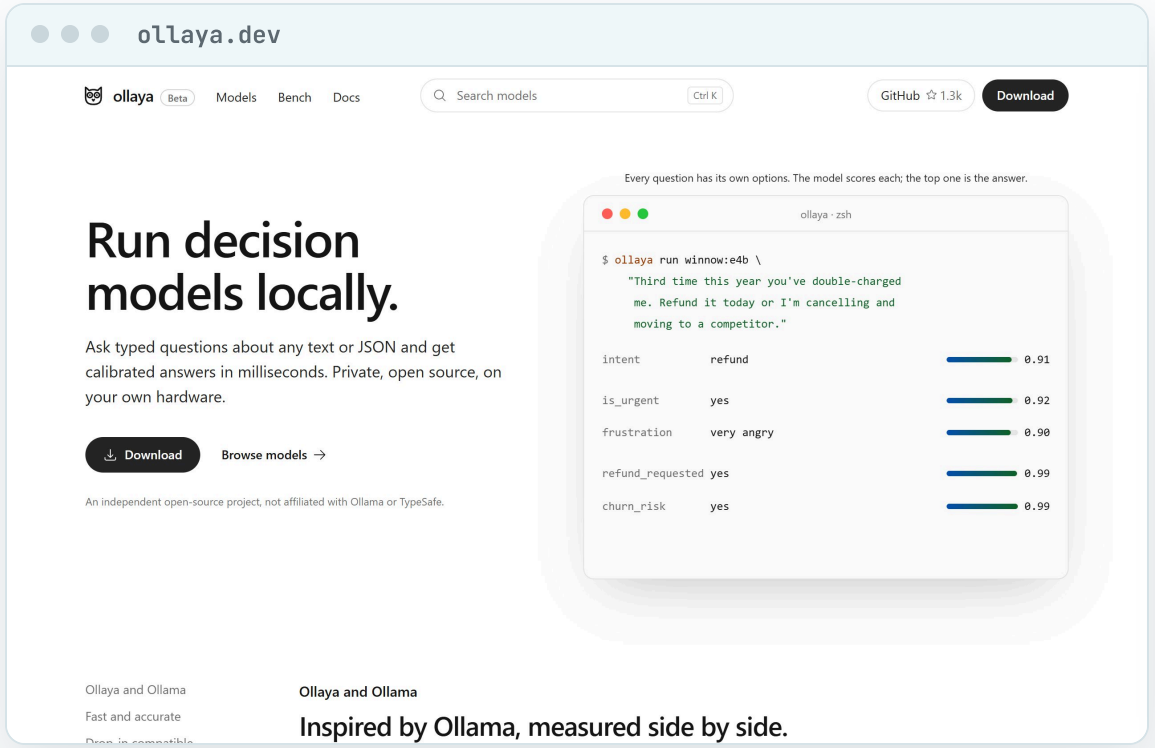
Ollaya | Laya | /api/decide | Language router | Presets

KGP Talkie

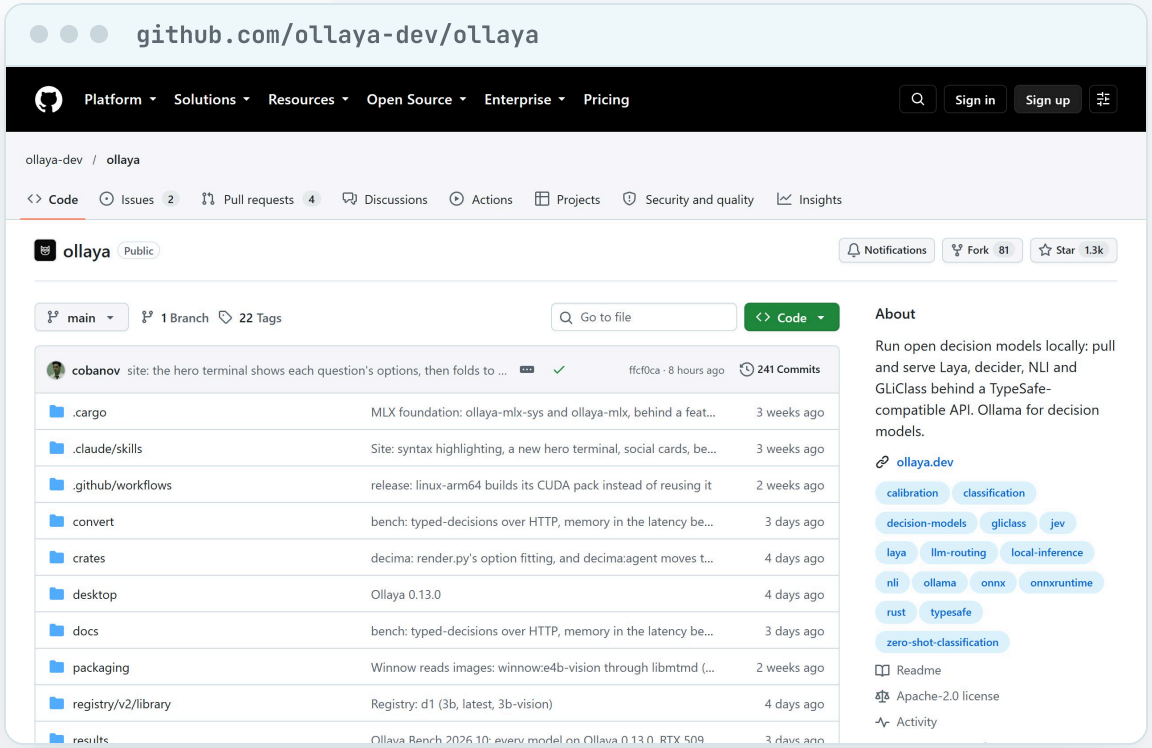
Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

Ollaya, Ollama for Decision Models



ollaya.dev: install, model library, docs



A Rust daemon and CLI, Apache-2.0

An independent project, not affiliated with Ollama or TypeSafe

Ollama vs Ollaya: What Changes

	Ollama	Ollaya
what it serves	text models	**decision models**
you send	a prompt	**a state + typed questions**
you get back	generated text, token by token	**probabilities per option, no text**
native endpoint	`/api/generate, /api/chat`	`/api/decide`
default port	`11434`	`11435`
model store	`~/.ollama/models`	`~/.ollaya/models`

same CLI verbs

`pull` `run` `serve` `list` `ps` `show` `stop` `rm` `cp` `create -f Modelfile`

THE PROJECT'S OWN COMPARISON TABLE

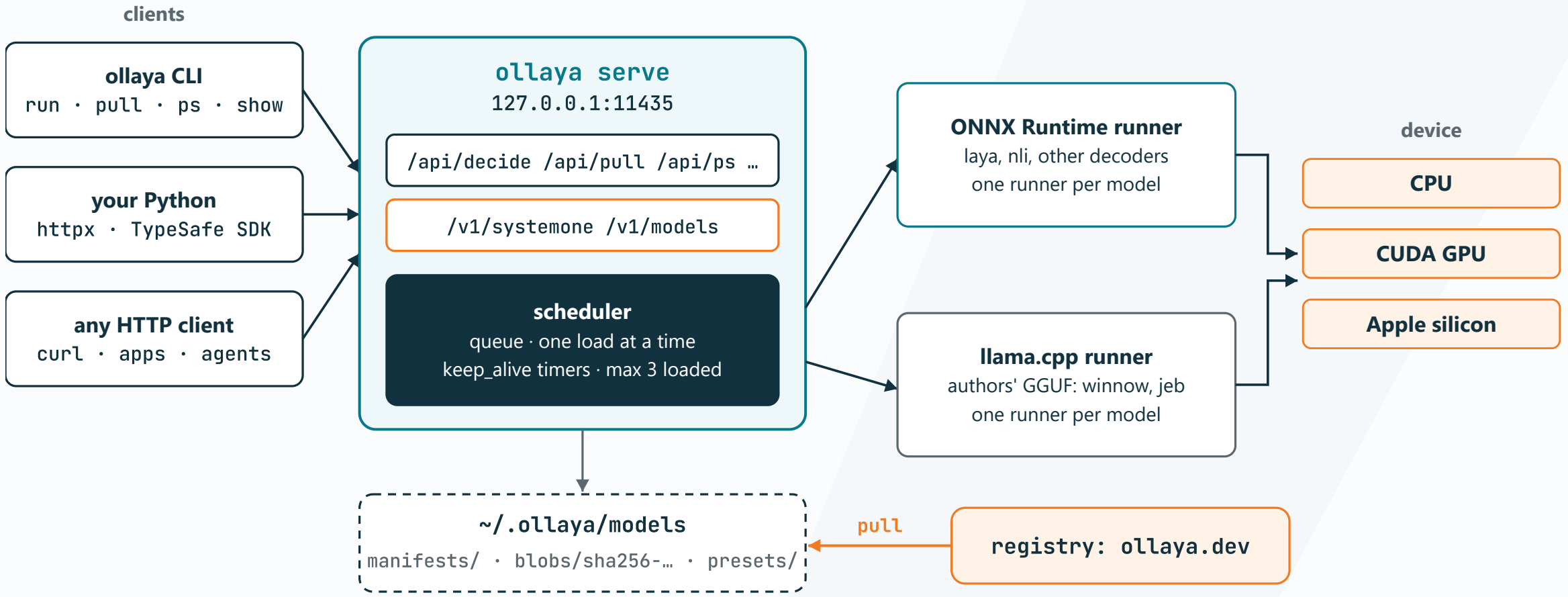
Ollaya Next to Ollama

ollaya.dev/docs/faq · Ollaya and Ollama

	Ollaya	Ollama 0.35
Models	21 families: encoders (`laya` , `nli` , `gliclass` , `von` , `decima`) and decoders (`winnow` , `clef` , `kev` , `decider` , `nimble` , `jeb` , `jeeves` , `cygnet` , `snap` , `arbiter` and more)	Nimble (9B) and Tev1 (4B, 0.8B)
Encoders	Read every question in one forward pass. `laya:en` answers five questions in 8 to 10 ms on an RTX 4090	Not supported
Probabilities	Calibrated with each model's fitted temperatures, which you can refit on your own data in a Modelfile	Softmax of the raw label scores. Ollama documents `confidence` as uncalibrated
Limits	TypeSafe's: 1 to 256 questions, 2 to 255 options, 2 to 10 score levels	1 to 64 questions, 2 to 26 options and score levels, a 64 KiB request body
API	TypeSafe's `/v1/systemone` , `/v1/decisions` and `/v1/models` ; `/api/decide` with routing and timings; an MCP server	`/v1/systemone`
Routers	`laya` picks the English or the multilingual model for each request	None
Weights	Pulled unmodified from the author's Hugging Face repository, pinned to a commit and checked by sha256	Converted to GGUF and served from Ollama's registry

Laya itself reads about 125 options (laya:en) or 250 (laya:multilingual); 255 is the API limit.

How Ollaya Is Built



What ollaya pull Does

you type `laya` → resolves to `ollaya.dev/library/laya:latest`

registry host	namespace	model	tag
ollaya.dev	library	laya	latest

pulling manifest	which files make this model
pulling 8d32a80bb199 …	each blob, resumable, shared blobs download once
verifying sha256 digest	a blob that does not match is thrown away
laya:en, laya:multilingual	a router also pulls every model it routes to
writing manifest	written last: a crash never leaves half a model
success	now listed by ollaya list

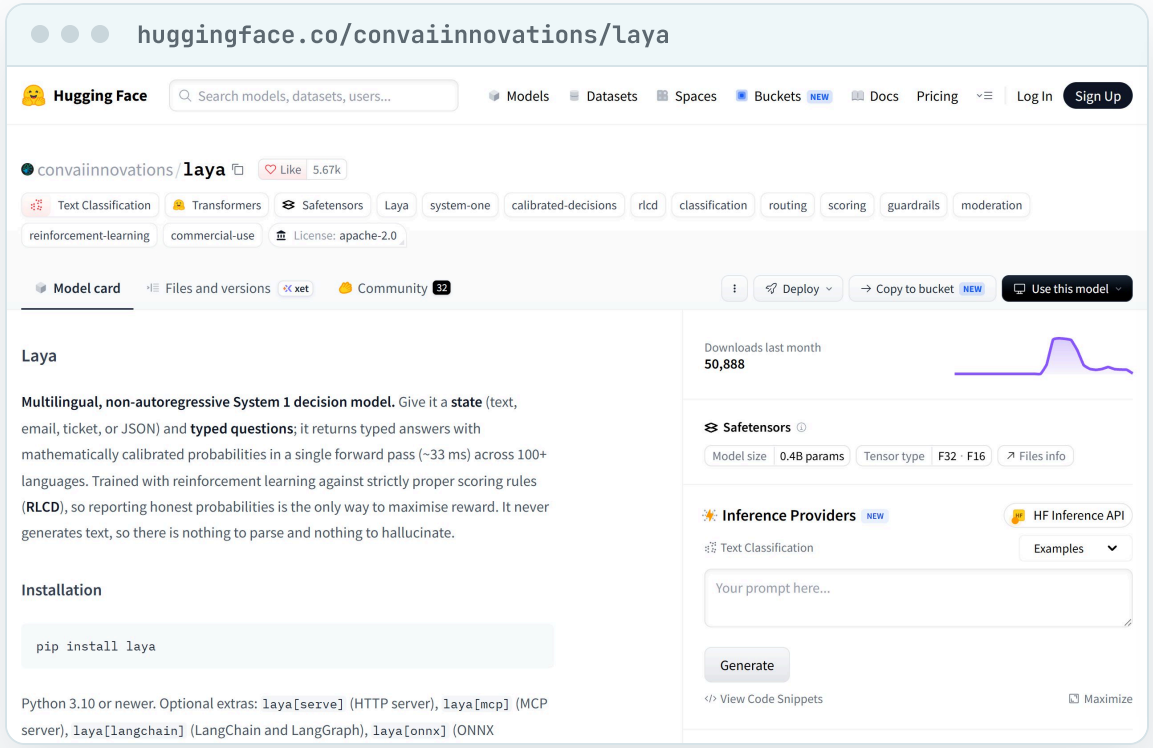
~/.ollaya/models

manifests/
ollaya.dev/library/laya/
latest en multilingual

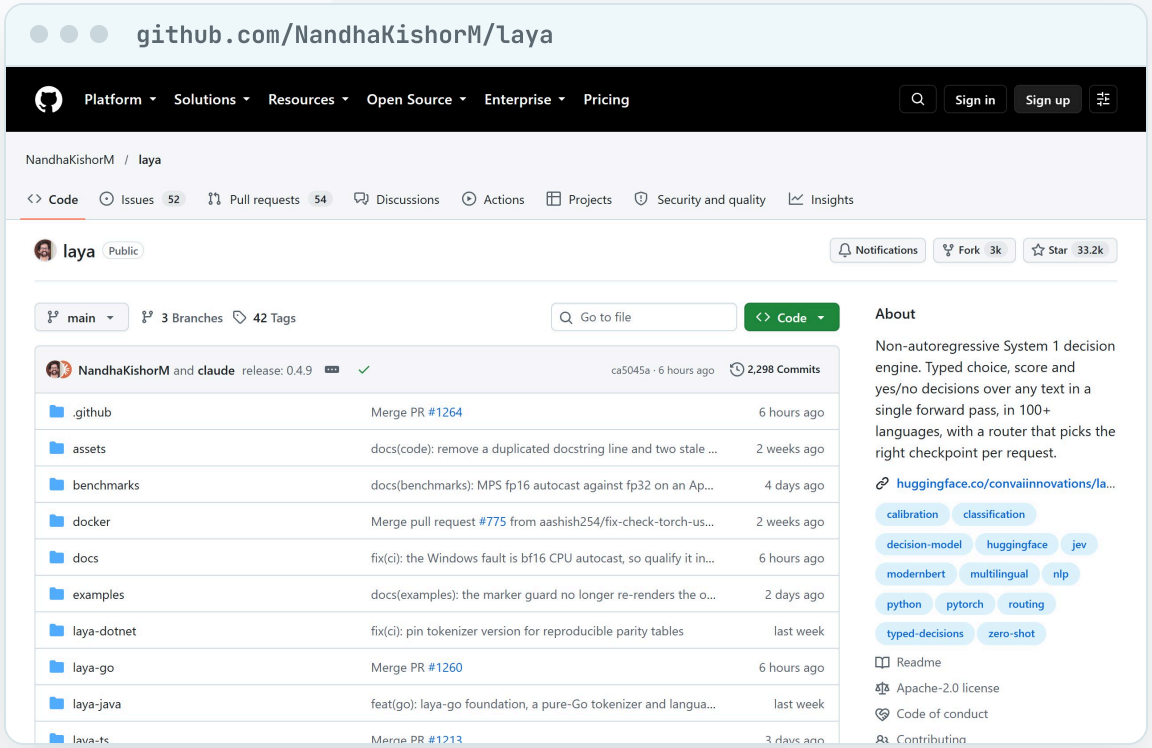
blobs/
sha256-… weights (ONNX)
sha256-… tokenizer
sha256-… config, license

presets/

Laya: Open Decision Models



Model card: checkpoints, RLCD training, calibration



The Python library, benchmarks and the router

Ollaya pulls the same weights, pinned by commit and sha256

Laya's Two Models: English and Multilingual

laya
a router
no weights
routes by script

laya:en — English

encoder: ModernBERT-large

421M parameters

512-token context

vocabulary 50,368 tokens

laya:multilingual — 100+ languages

encoder: mmBERT-base

322M parameters

1,024-token context

vocabulary 256,000 tokens

both checkpoints

Convai Innovations, Apache 2.0

the same decision head design

choice · score · noul

ONNX, fp16 graph on a GPU,

fp32 graph on a CPU

an act head, not yet a good gate

fitted temperatures: laya:en only

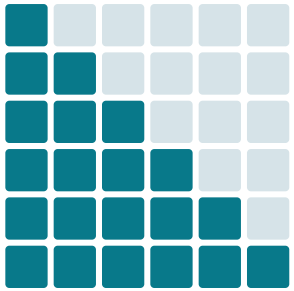
laya:multilingual: temperature 1.0

both over-confident until you refit

Encoder vs Decoder

Decoder

chat models, Jev



attention mask

each token sees only earlier tokens

reads the prompt, then writes

one new token per step

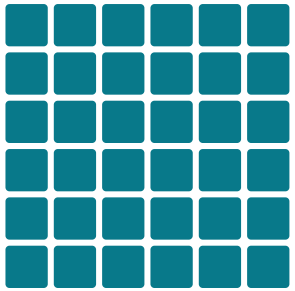
answer = text you parse

`prompt` → `tok` → `tok` → `tok` → `…`

a loop: the model runs again for every output token

Encoder

Laya



attention mask

every token sees every token

reads all tokens at once

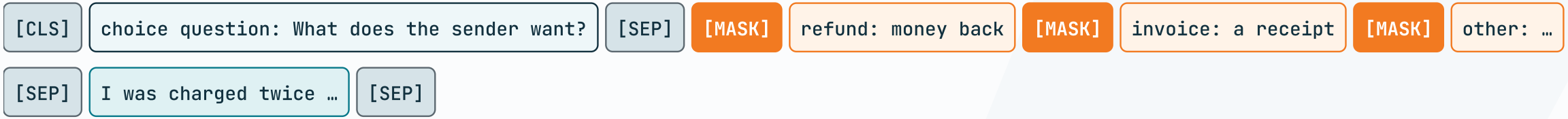
writes nothing

answer = vectors read by a head

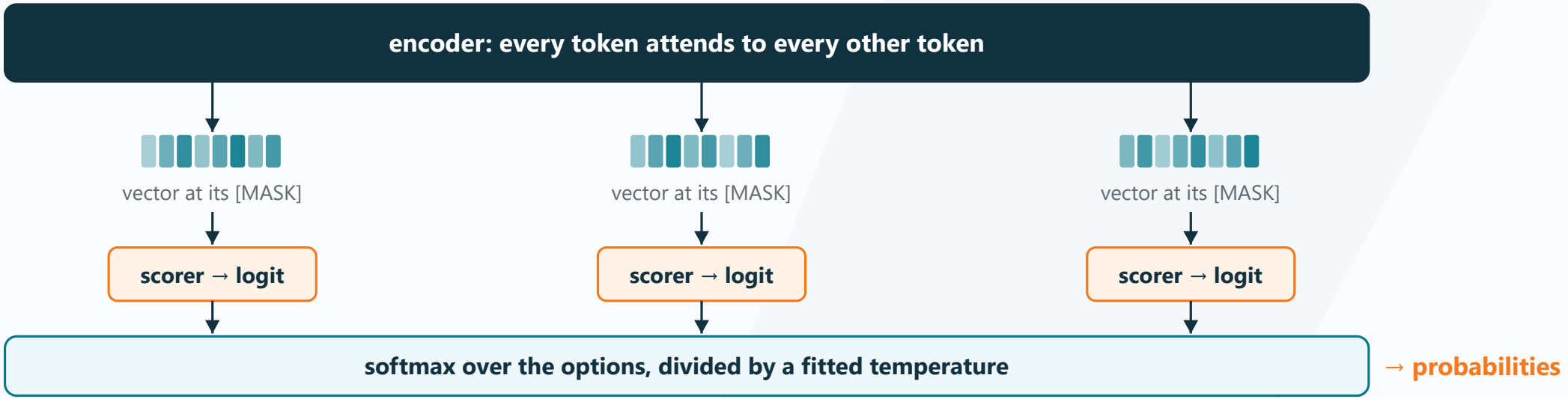
`state + questions` → `one pass` → `probabilities`

no loop: one forward pass answers every question

How Laya Answers a Choice



one row of tokens: the question, each option behind a [MASK] marker, then the state



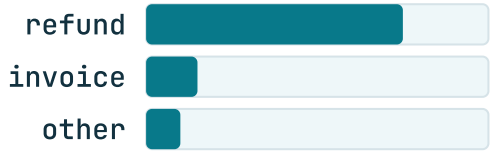
The options are part of the input, so new labels need no retraining.

How Laya Answers Score and Noul

choice

options = the criteria labels

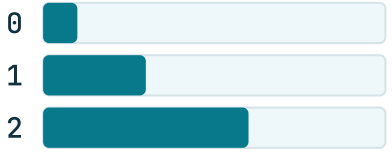
pick the top option



score

options = the levels 0, 1, 2

score = Σ i · pᵢ



noul

options = false, true

noul = P(true)



one forward pass

3 rows, padded
into one batch
usage = all their tokens

Score, worked out

`p = (0.1, 0.3, 0.6) for levels 0, 1, 2`

`score = 0·0.1 + 1·0.3 + 2·0.6 = 1.5`

between level 1 and level 2

Parts of the ollaya run Command


```
ollaya run laya "Where is my order 20931?" --preset triage
```

model — state — question set

```
intent              label          ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
is_urgent           yes / no       ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
frustration         level / 3      ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
refund_requested    yes / no       ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
churn_risk          yes / no       ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓
```

one line per question: the answer and its confidence bar

`--verbose`

every option's probability, the model that answered, timings

`--format json`

the full /api/decide response

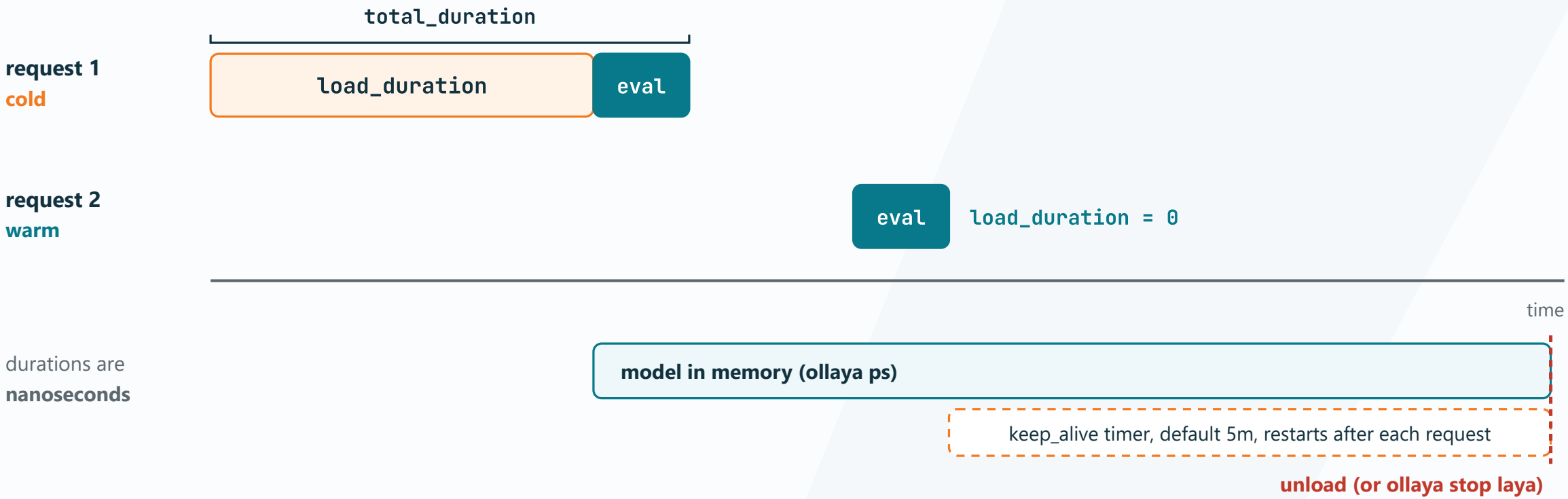
`--questions q.json`

your own questions instead of a preset

`--keepalive 10m`

how long the model stays in memory

Cold and Warm Requests



Ollaya's Two APIs: Native and TypeSafe

ollaya serve
:11435
one model store
one scheduler

POST /api/decide

native: httpx, curl, the CLI

model · answers · usage

+ routing: router, model, route, reason
+ total / load / eval_duration
+ state_truncated, done_reason

request extras: keep_alive, preset, extras

POST /v1/systemone

TypeSafe-compatible: the SDK

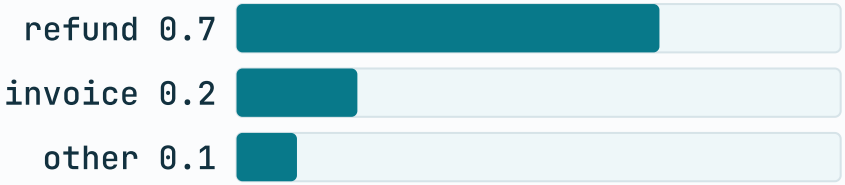
model · answers · usage

nothing else: wire-identical to TypeSafe
also /v1/decisions and /v1/models
other fields in the request are ignored

hosted Jev code runs unchanged

Two Confidence Formulas Compared

One answer, probabilities stated:



confidence

`(K · p_max - 1) / (K - 1)`

K = 3 options, p_max = 0.7

(3 · 0.7 - 1) / (3 - 1)

= 1.1 / 2

= 0.55

TypeSafe's formula: how far the top option is above a uniform guess

laya.confidence

`1 - H(p) / ln K`

H = -(0.7 ln 0.7 + 0.2 ln 0.2 + 0.1 ln 0.1)

H = 0.250 + 0.322 + 0.230 = 0.802

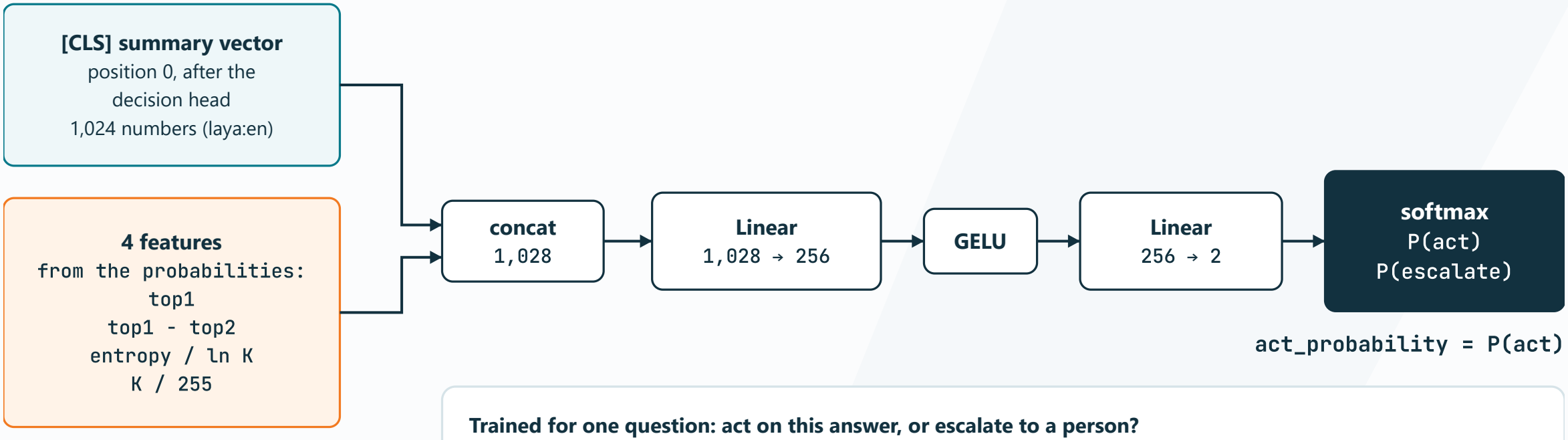
1 - 0.802 / 1.099

= 0.27

entropy based: low unless the whole distribution is peaked

Thresholds do not transfer: tune a gate on the number you will use, per model.

The Act Head: Built, Not Yet Useful



Trained for one question: act on this answer, or escalate to a person?

Today it reads about 1.0 for almost every input, so it is not a usable signal.

Gate on confidence (or laya.confidence) instead. laya.* comes from "extras": ["laya"].

Pointing the Jev Client at Ollaya

HOSTED

```python
from typesafe_sdk import TypeSafeClient

client = TypeSafeClient()
# URL and key come from .env

client.system_one(state, QUESTIONS)
```

LOCAL: THREE ARGUMENTS CHANGE

```python
from typesafe_sdk import TypeSafeClient

client = TypeSafeClient(
    base_url="http://localhost:11435",
    api_key="local",
    model="laya")
client.system_one(state, QUESTIONS)
```

POST /v1/systemone

openrouter.ai/api
hosted Jev, per-token price

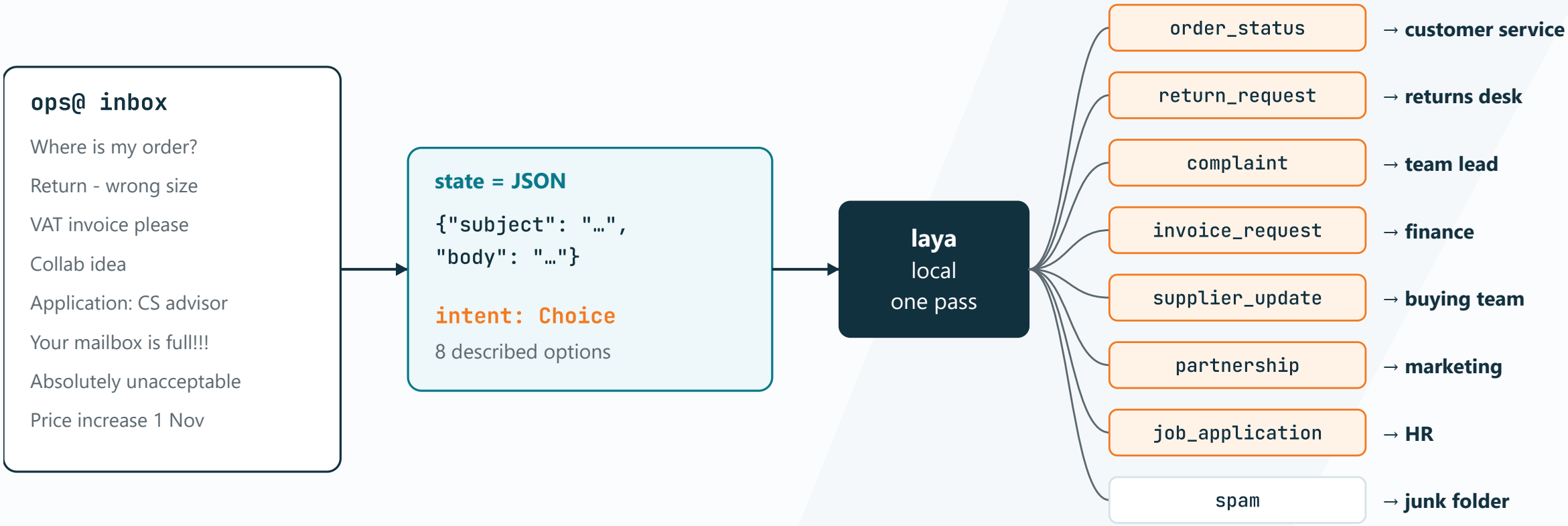
or

localhost:11435
Ollaya + Laya, your machine

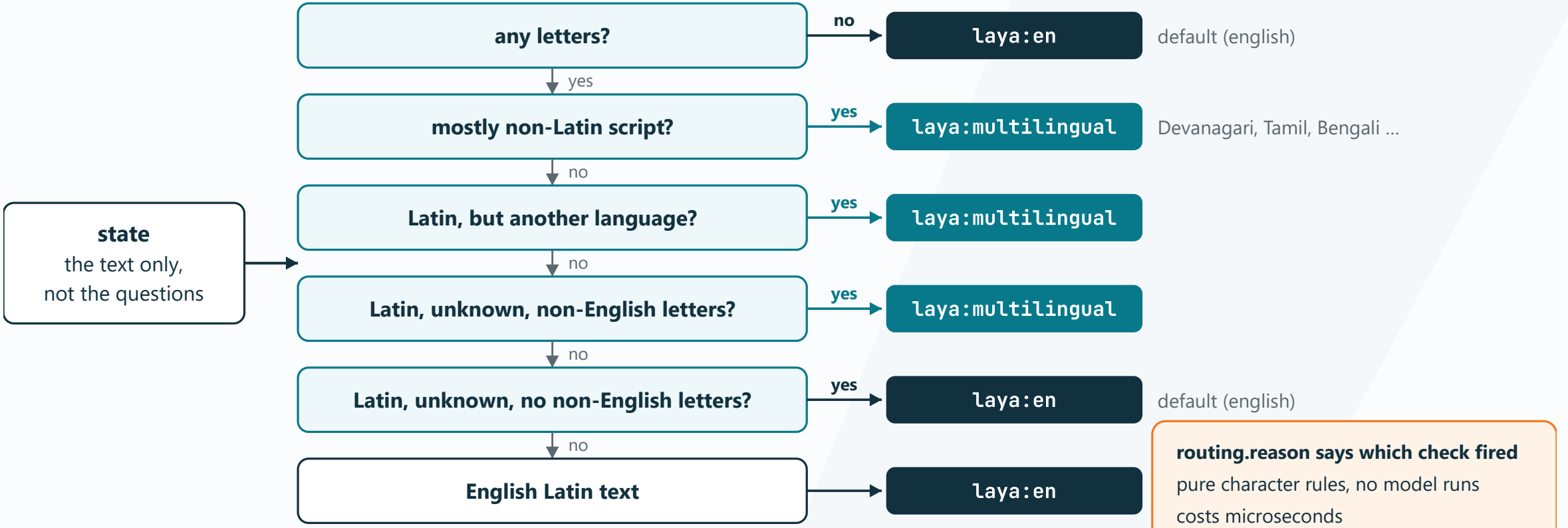
same request body
same answer objects

Ollaya ignores the key unless `OLLAYA_API_KEY` is set, but the SDK needs a non-empty one.

Use Case: Routing an Inbox



How Ollaya Picks a Model by Language



routing.reason says which check fired
pure character rules, no model runs
costs microseconds
"model": "laya:en" skips it

Hindi Through Two Tokenizers

खाना ठंडा आया khana thanda aaya “the food came cold”

ModernBERT-large

laya:en · 50,368-token vocabulary

?	?	◌ा	न	◌ा	?	?	◌ं	?	?	◌ा	?	?	य	◌ा

15 pieces, most are half a character

mmBERT-base

laya:multilingual · 256,000-token vocabulary

_खा	ना	_ठ	◌ंड	◌ा	_आया

6 pieces, real Hindi word parts

either tokenizer

the romanized line, Latin script

kh	ana	_th	anda	_a	aya

6 pieces, but spelled sounds, not English words

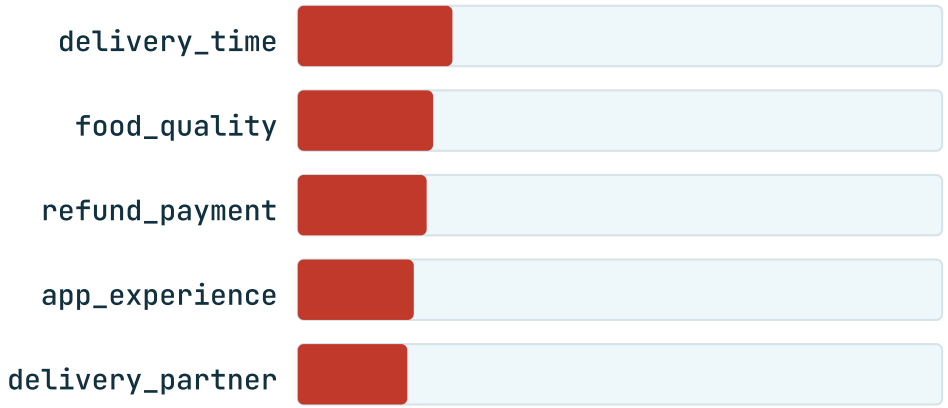
? = a byte fragment: one Devanagari character split into pieces the English vocabulary never learned

_ = this piece starts a new word

English Model on Hindi Text

laya:en reads Hindi

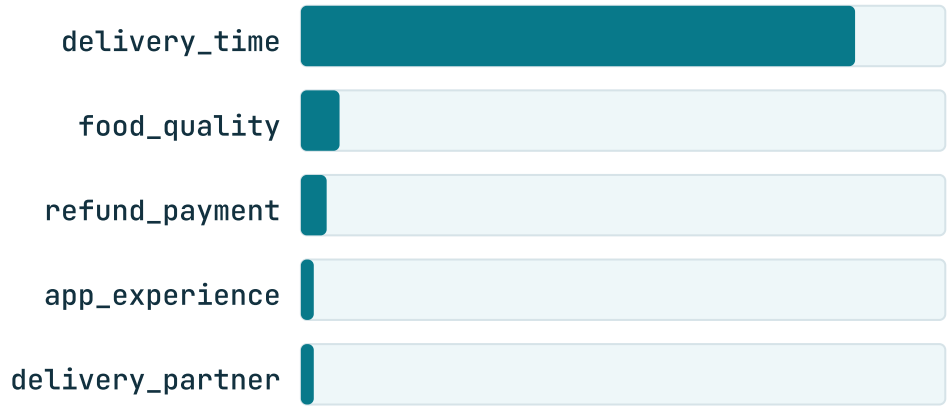
Devanagari → byte fragments → vectors with little meaning



close to a uniform guess: whichever label wins, the confidence stays low

laya:multilingual reads Hindi

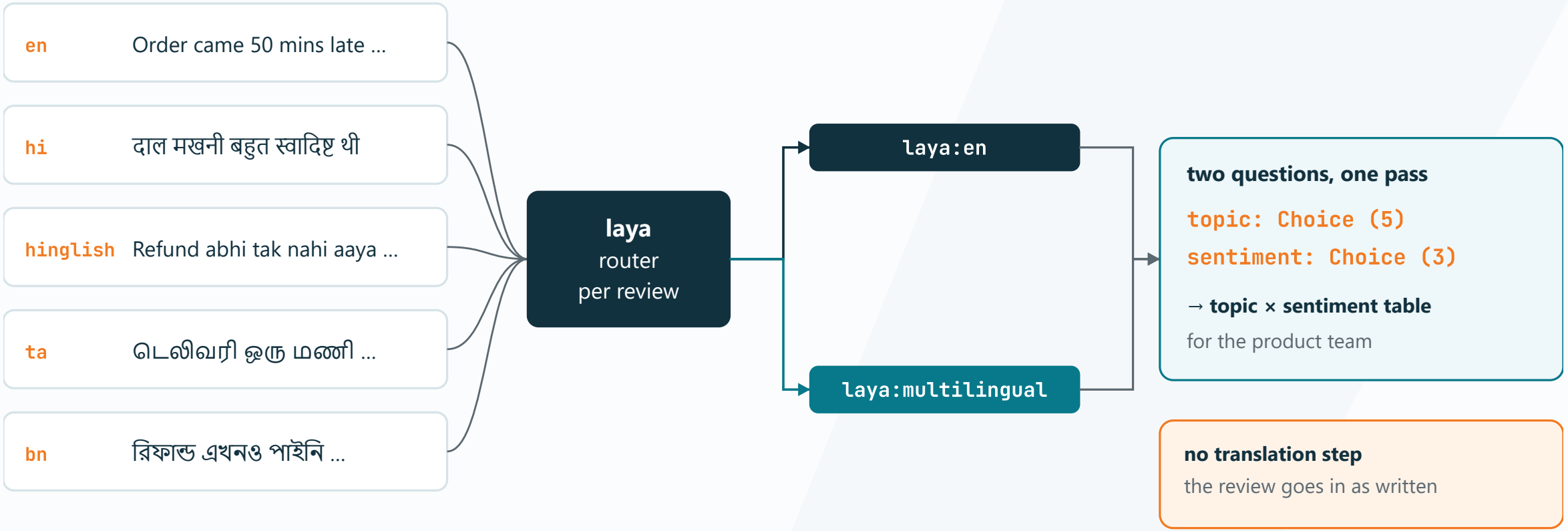
Devanagari → Hindi word pieces → vectors trained on Hindi



one option stands out: the answer depends on the text

illustration: bar shapes, not measured values

Use Case: Feedback in Five Languages



Latency vs Throughput

the same 200 requests, sent at once with asyncio.gather

hosted Jev
many servers



many requests in flight at once, on many machines

Ollaya + Laya
one runner per model



no network, but one forward pass at a time: the queue drains in order

Latency (one request) and throughput (many requests) are different questions: measure both.

Local or Hosted?

	LOCAL: OLLAYA + LAYA	HOSTED: JEV
Privacy	text never leaves your machine	text goes to the provider
Cost	no per-token price: your hardware and power	per million input tokens, no servers to run
One request	no network round trip	a round trip over the internet
Many requests	one queue per model, your hardware's limit	scales out on their servers
Ops	you install, update, monitor, size the machine	the provider runs it
Quality	depends on the task: measure on your data	depends on the task: measure on your data

CPU or GPU

OLLAYA_DEVICE
`auto (default)`
`cpu · cuda · cuda:1`

CUDA GPU

loads the fp16 graph

weights in VRAM (size_vram in /api/ps)

fastest per request

NVIDIA driver R527+ on Windows, R525+ on Linux

CPU

loads the fp32 graph

weights in RAM, size_vram = 0

no GPU needed: encoders stay quick

slower first load, slower per request

same model file

laya:en carries both graphs: precision F16/F32 in ollaya show

fp16 and fp32 agree on almost every answer; near-ties (top two options within 0.01) can flip

ollaya ps shows the device and precision actually loaded

Vulkan on Windows is for GGUF models only, laya is ONNX

Presets and Modelfiles

six built-in presets

`triage`	`intent, is_urgent, frustration, refund_requested, churn_risk`
`email`	`category, is_spam, is_phishing, urgency, needs_reply`
`guard`	`jailbreak, prompt_injection, sensitive_data, harm_severity, topic`
`moderation`	`toxic, harassment, threat, spam, severity`
`router`	`difficulty, domain, needs_tools, is_sensitive`
`agent`	`action, on_task, risk, destructive`

custom preset

a named question set, works with any model

presets/inbox.json in the model store

Modelfile → a model with questions inside

```
FROM laya
QUESTIONS ./inbox.json
```

→

```
ollaya create
store-inbox -f Modelfile
```

`manifest of store-inbox:latest`

router layer	from laya, reused
license layer	from laya, reused
questions: intent	one new layer, a few KB

laya holds no weights. store-inbox routes to the same laya:en and laya:multilingual, so none are copied.

```
ollaya run store-inbox "…" needs no --preset
```

SECTION 04 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie